SOLAR▸SWARM

Technical Reference
Martin Pham | Perth, AU

Memory & Database Reference

Overview

All persistent state is stored in a single SQLite file (`swarm.db` by default, configurable via `DATABASE_PATH`). The database uses WAL (Write-Ahead Logging) journal mode for concurrent access safety.

Three memory tiers serve different access patterns:

TIER	STORAGE	ACCESS SPEED	PURPOSE
Hot	SQLite (`swarm.db`)	Sub-second	Live state: experiments, tasks, leads, signals
Warm	JSON files (`memory/knowledge/`)	~10ms	Pattern recognition, retrospective analysis
Cold	SQLite (`cold_ledger` table)	Write-once	Immutable audit trail; never updated

Full SQLite Schema

Schema defined in `memory/database.py` `init_db()`. All tables created with `IF NOT EXISTS`.

`experiments`

Core experiment tracking table.

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	INTEGER	PK AUTOINCREMENT	
created_at	TEXT	DEFAULT datetime('now')	
idea_text	TEXT	NOT NULL	Full experiment description
vertical	TEXT		e.g. solar_australia
bucket	TEXT	CHECK: exploit/explore/moonshot	Portfolio bucket
confidence_score	REAL		0–10 composite score
devil_score	REAL		Red team score 1–10
kelly_fraction	REAL		Calculated Kelly fraction
budget_allocated	REAL	DEFAULT 0	AUD allocated
status	TEXT	DEFAULT 'pending', CHECK enum	pending/approved/running/complete/killed/rejected
revenue_generated	REAL	DEFAULT 0	Actual revenue from this experiment
roi	REAL		Return on investment
learnings	TEXT		Free-text learnings
failure_mode	TEXT		Why it failed
approved_by	TEXT		'auto' or username
approved_at	TEXT		ISO timestamp
completed_at	TEXT		ISO timestamp
paid_spend_activated	INTEGER	DEFAULT 0	72hr explore protocol flag
explore_phase	TEXT		Current phase name

`task_queue`

Work queue for Tier 3 workers.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
job_type	TEXT NOT NULL	e.g. research_prospect, content_ad_copy
priority	INTEGER	1=highest, 10=lowest. Tier 2 uses 3–7
context_payload	TEXT	JSON dict of task parameters
assigned_to	TEXT	Worker identifier
status	TEXT	queued/running/complete/failed
output	TEXT	JSON result from worker
completed_at	TEXT	
tier	INTEGER	DEFAULT 3

`pheromone_signals`

Reinforcement signals from workers that bias future experiment generation.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
signal_type	TEXT	POSITIVE/NEGATIVE/NEUTRAL
topic	TEXT	e.g. solar_prospecting, lead_quality
vertical	TEXT	e.g. solar_australia
strength	REAL	0.0–1.0
channel	TEXT	Source channel
experiment_id	INTEGER	Associated experiment
decay_factor	REAL	DEFAULT 1.0; reduced by apply_pheromone_decay()

`cold_ledger`

Append-only audit trail. Records are never updated or deleted.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
event_type	TEXT NOT NULL	e.g. EXPERIMENT_CREATED, LEAD_QUALIFIED
event_data	TEXT	JSON dict of event details
experiment_id	INTEGER	Associated experiment if applicable
agent_id	TEXT	Which agent triggered this
human_involved	INTEGER	0/1

`leads`

Solar lead records.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
source	TEXT	ghl_webhook/manual/form
name	TEXT	
phone	TEXT	
email	TEXT	
suburb	TEXT	
state	TEXT	AU state code
homeowner_status	TEXT	owner/renter/unknown
monthly_bill	REAL	AUD per month
roof_type	TEXT	tile/colorbond/flat/metal
roof_age	INTEGER	Years
qualification_score	REAL	1–10 AI score
score_reason	TEXT	2-sentence explanation
recommended_action	TEXT	call_now/nurture/disqualify
pipeline_stage	TEXT	GHL stage name
status	TEXT	DEFAULT 'new'
contacted_at	TEXT	When first contacted
converted_at	TEXT	When converted to client
client_account	TEXT	Solar company client identifier
notes	TEXT	Appended call notes, outcomes

`circuit_breaker_log`

Circuit breaker events.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
triggered_at	TEXT	
level	TEXT	yellow/orange/red
reason	TEXT	Human-readable reason
consecutive_failures	INTEGER	Failure streak at trigger time
budget_burn_rate	REAL	Burn rate at trigger time
resolved_at	TEXT	NULL if still active
resolved_by	TEXT	Who reset it

`message_bus`

Inter-agent communication bus.

COLUMN	TYPE	DESCRIPTION
id	INTEGER PK	
created_at	TEXT	
msg_id	TEXT UNIQUE	e.g. msg_abc123def456
from_agent	TEXT NOT NULL	Sender identifier
to_queue	TEXT NOT NULL	Destination queue name
msg_type	TEXT NOT NULL	TASK/REPORT/ALERT/ACK/KILL/QUERY/RESPONSE
priority	TEXT	CRITICAL/HIGH/NORMAL/LOW
payload	TEXT	JSON dict
reply_to	TEXT	msg_id this responds to
ttl_cycles	INTEGER	DEFAULT 3; expire after N cycles
requires_ack	INTEGER	0/1
status	TEXT	queued/processing/complete/failed/expired
acked_at	TEXT	
completed_at	TEXT	

`research_findings`

Results from the research engine.

COLUMN	TYPE	DESCRIPTION
research_id	TEXT UNIQUE	Unique research request ID
research_type	TEXT	prospect/market/competitive/technical
query	TEXT	The research query
requested_by	TEXT	Agent that requested it
status	TEXT	pending/in_progress/complete/failed
findings	TEXT	JSON research results
confidence	REAL	0.0–1.0
sources_count	INTEGER	Number of sources consulted
opportunities_found	INTEGER	Count of opportunities identified
expires_at	TEXT	Cached findings expiry
completed_at	TEXT	

`kg_entities`

Knowledge graph entities.

COLUMN	TYPE	DESCRIPTION
entity_id	TEXT UNIQUE	e.g. ent_abc123def
entity_type	TEXT	company/person/tool/market/location
name	TEXT	Entity name
properties	TEXT	JSON dict of attributes
confidence	REAL	0.0–1.0
source	TEXT	Where discovered
mention_count	INTEGER	How many times seen
first_seen	TEXT	
last_seen	TEXT	

`kg_relationships`

Knowledge graph edges between entities.

COLUMN	TYPE	DESCRIPTION
rel_id	TEXT UNIQUE	e.g. rel_xyz789
from_entity	TEXT	Source entity_id
to_entity	TEXT	Target entity_id
rel_type	TEXT	USES/COMPETES_WITH/LOCATED_IN/EMPLOYS/PARTNERS_WITH/REFERS
properties	TEXT	JSON dict
confidence	REAL	

`collection_sources`

Registry of all data collection sources.

COLUMN	TYPE	DESCRIPTION
source_id	TEXT UNIQUE	e.g. cec_registry_wa
name	TEXT	Human-readable name
source_type	TEXT	web_scrape/api_poll/social/price_monitor
url_template	TEXT	URL pattern
config	TEXT	JSON configuration
collection_frequency	TEXT	daily/hourly/weekly
frequency_hours	INTEGER	DEFAULT 24
priority	TEXT	NORMAL/HIGH/LOW
active	INTEGER	1=enabled
last_collected	TEXT	
health_status	TEXT	healthy/degraded/dead
error_count	INTEGER	Consecutive error count

`collected_data`

Raw data from all collection sources.

COLUMN	TYPE	DESCRIPTION
source_id	TEXT	References collection_sources
source_type	TEXT	Inherited from source
record_type	TEXT NOT NULL	Data category
record_key	TEXT	Optional dedup key
raw_data	TEXT	Original payload JSON
normalized_data	TEXT	Processed payload JSON
quality_score	REAL	0.0–1.0
dedup_hash	TEXT	Fingerprint for deduplication
processed	INTEGER	0/1
normalized	INTEGER	0/1

`time_series`

Metric recording for trend detection.

COLUMN	TYPE	DESCRIPTION
recorded_at	TEXT	
series_name	TEXT	e.g. ctr, cpl, lead_volume
entity_id	TEXT	Associated experiment or source
value	REAL	Metric value
unit	TEXT	e.g. AUD, percent, count
tags	TEXT	JSON dict

`opportunities`

Opportunities discovered by scout/research agents.

COLUMN	TYPE	DESCRIPTION
opportunity_id	TEXT UNIQUE	e.g. opp_abc123
opp_type	TEXT	prospect/market_gap/partnership
title	TEXT	
description	TEXT	
estimated_monthly_revenue_aud	REAL	
effort_score	REAL	1–10
speed_score	REAL	1–10
risk_score	REAL	1–10
overall_score	REAL	
status	TEXT	new/researching/queued/active/passed/killed
source_agent	TEXT	
research_id	TEXT	
experiment_id	INTEGER	
evidence	TEXT	JSON dict

`ab_tests`

A/B test tracking.

COLUMN	TYPE	DESCRIPTION
test_id	TEXT UNIQUE	e.g. abt_abc123
name	TEXT	
hypothesis	TEXT NOT NULL	
variant_a	TEXT	JSON config for control
variant_b	TEXT	JSON config for treatment
metric	TEXT	conversion_rate/reply_rate/booking_rate
status	TEXT	running/complete/inconclusive
a_impressions / b_impressions	INTEGER	
a_conversions / b_conversions	INTEGER	
a_clicks / b_clicks	INTEGER	
winner	TEXT	a/b/no_winner
winner_stats	TEXT	JSON stats dict
completed_at	TEXT	

`call_logs`

Voice AI call records.

COLUMN	TYPE	DESCRIPTION
call_id	TEXT UNIQUE	Retell call ID
client_id	TEXT	Solar company client
from_phone	TEXT	Caller number
to_phone	TEXT	Called number
agent_id	TEXT	Retell agent ID
status	TEXT	started/active/complete/failed
duration_seconds	INTEGER	
recording_url	TEXT	
outcome	TEXT	booked_assessment/callback_requested/etc
lead_score	REAL	
summary	TEXT	GPT-4o call summary
transcript_turns	INTEGER	

`api_usage`

API cost tracking.

COLUMN	TYPE	DESCRIPTION
service	TEXT	openai/retell/elevenlabs
operation	TEXT	chat.completion/phone_call/tts
model	TEXT	gpt-4o/etc
units	REAL	Tokens, seconds, characters
unit_type	TEXT	tokens/seconds/characters
cost_usd	REAL	Estimated USD cost
call_id	TEXT	Associated voice call
client_id	TEXT	Associated solar client
metadata	TEXT	JSON

`crm_cache`

CRM data cached by `crm_sync` every 30 minutes.

COLUMN	TYPE	DESCRIPTION
cached_at	TEXT	
cache_key	TEXT UNIQUE	e.g. contact_abc123, pipeline_stage_xyz, metrics_summary
cache_value	TEXT	JSON serialised CRM data

Cache TTL: Data is refreshed every 30 minutes by the `crm_sync` scheduler job. The dashboard API reads from this cache rather than hitting the CRM directly on every request.

Other Tables

TABLE	PURPOSE
`users`	Multi-user auth (email, password_hash, role: owner/admin/client)
`auth_tokens`	JWT revocation list
`company_profiles`	One row per solar SME client (name, ABN, contact_email, retell_agent_id, etc.)
`api_keys`	Client embed / webhook auth keys
`app_settings`	Runtime config overrides (key, value, category)
`email_logs`	Email processing records (intent, score, draft_reply_queued)

How Migrations Work (`_apply_migrations`)

The `_apply_migrations()` function in `memory/database.py` runs on every startup, immediately after `init_db()`. It adds columns that exist in application code but may be missing from older database schemas.

Each migration is an `ALTER TABLE ADD COLUMN` statement. SQLite will raise an exception if the column already exists, which is caught and silently ignored.

```
migrations = [
    ("ab_tests", "name TEXT"),
    ("ab_tests", "experiment_id INTEGER"),
    ("kg_entities", "properties TEXT"),
    ("experiments", "paid_spend_activated INTEGER DEFAULT 0"),
    ("experiments", "explore_phase TEXT"),
    # ... etc
]
```

This design means the database schema is always at least as current as the code, with no manual migration steps required.

Hot Memory API Reference

File: `memory/hot_memory.py`

Experiments

```
get_active_experiments()                    # status IN ('approved','running')
get_pending_experiments()                   # status = 'pending', sorted by confidence DESC
get_experiment(experiment_id)               # single row dict
update_experiment_status(id, status, extra) # update status + optional extra fields
get_budget_used_this_week()                 # float AUD
get_consecutive_failures()                  # int — current failure streak
```

Task Queue

```
enqueue_task(job_type, context, priority=5, tier=3)  # returns task_id int
get_next_task(tier=None)                              # highest-priority queued task
complete_task(task_id, output)
fail_task(task_id, reason)
```

Pheromone Signals

```
post_pheromone(signal_type, topic, strength, vertical, channel, experiment_id)
get_active_pheromones(topic=None)    # decay_factor > 0.01
apply_pheromone_decay()              # 50% decay to signals older than 7 days
```

Circuit Breaker

```
get_circuit_breaker_state()   # most recent unresolved log row; {level, active, ...}
get_swarm_summary()           # {active_experiments, pending_approval, budget_used_aud, ...}
```

Warm Memory API Reference

File: `memory/warm_memory.py`

Files stored in `memory/knowledge/`:

```
save_experiment_outcome(experiment_id, idea_text, outcome)
save_learning(topic, insight, source, confidence=0.7)
get_learnings_for_topic(topic)     # sorted by confidence DESC
get_all_learnings()                # full list
get_winning_patterns()             # complete experiments with positive ROI
get_experiment_history(limit=50)   # sorted by recorded_at DESC
update_vertical_knowledge(vertical, data)
get_vertical_knowledge(vertical)
```

Cold Ledger API Reference

File: `memory/cold_ledger.py`

```
log_event(event_type, event_data, experiment_id, agent_id, human_involved)
log_experiment_created(experiment_id, idea_text, confidence, agent_id)
log_experiment_approved(experiment_id, approved_by, budget)
log_experiment_killed(experiment_id, reason, agent_id)
log_circuit_breaker(level, reason, data)
log_lead_qualified(lead_id, score, action)
log_pheromone_signal(signal_type, topic, strength, experiment_id)
get_recent_events(limit=50)
get_events_by_type(event_type, limit=100)
```

Knowledge Graph API Reference

File: `storage/knowledge_graph.py`

```
upsert_entity(name, entity_type, properties, source, confidence)  # create or update; returns entity_
upsert_relationship(from_id, to_id, rel_type, properties, confidence)  # returns rel_id
get_entity(name, entity_type=None)
get_neighbors(entity_id, rel_type=None)
search_entities(entity_type, limit=20)  # sorted by mention_count DESC
get_graph_summary()  # {entities: {type: count}, relationships: {type: count}}
```

Upsert pattern: If an entity with the same `name` and `entity_type` already exists, its `properties`, `confidence`, `last_seen`, and `mention_count` are updated. If not, a new entity with a generated `entity_id` is created.